팀1 — 공기압축기 예지보전 · 우선점검 대상 선정

K-뉴딜 아카데미 스마트팩토리 캡스톤 · 3~6회차 통합 실습

데이터	분석단위	규모	기간
`team1_compressor.csv`	압축기 1대의 특정 시각 운전기록 1건	1,800행 × 22열	2026년 7월 1일 ~ 7월 20일 (압축기 C01~C04 각 450건)

이 노트북에서 하는 일

데이터 탐색 → 시각화 → 전처리 → 머신러닝 모델 3종 → 성능평가 → 특성중요도

현장 상황

가상의 **생활가전 제조공장**입니다. 공기압축기 4대(C01~C04)가 생산설비에 압축공기를 공급합니다. 최근 **압력 저하, 토출온도 상승, 진동 증가** 같은 이상징후와 **비계획정지**가 간헐적으로 발생하고 있습니다.

정비 인력은 한정되어 있어서 4대를 동시에 분해점검할 수 없습니다. 그래서 보전 담당자는 **매일 교대 종료 전에 데이터를 보고, 다음 24시간 안에 우선 점검할 압축기를 고르는** 의사결정을 합니다.

압축공기는 "공장의 네 번째 유틸리티"라고 불립니다. 압축기가 갑자기 멈추면 공기를 쓰는 모든 라인이 함께 멈추므로, 한 대의 고장이 공장 전체의 생산손실로 번집니다. 반대로 멀쩡한 설비를 자주 분해점검하면 정비비와 가동손실이 늘어납니다. **"놓치는 비용(미탐)"과 "헛되게 점검하는 비용(오탐)"의 균형**이 이 프로젝트의 본질입니다.

핵심 질문

> **다음 24시간 안에 정비가 필요한 압축기는 어느 것이고, 어떤 항목(진동·냉각·필터)을 먼저 점검해야 하는가?**

판단 시점

> **교대 종료 직전 (지금까지의 센서값은 알고, 앞으로 24시간에 일어날 일은 모르는 시점)**

판단 시점이 중요한 이유: **그 시점에 알 수 없는 값**을 입력으로 쓰면 시험 답안지를 미리 본 것과 같아집니다(**데이터 누출**). 아래 8절에서 직접 걸러냅니다.

---

1. 준비 — 라이브러리와 한글 폰트

도구	역할
`pandas` (pd)	표 데이터 처리 (엑셀에 해당)
`numpy` (np)	숫자 계산
`matplotlib` (plt) · `seaborn` (sns)	그래프
`scikit-learn`	머신러닝

아래 셀은 **한 번만** 실행하면 됩니다.

```python
# [준비 셀] 내용을 자세히 이해할 필요는 없습니다. 실행만 하세요.
# 라이브러리 불러오기 + 한글 폰트 설정
import os, sys, glob, warnings
import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (confusion_matrix, classification_report, roc_curve,
                             roc_auc_score, recall_score, precision_score)
warnings.filterwarnings("ignore")
IN_COLAB = "google.colab" in sys.modules

# ------------------------------------------------------------------------
# 한글 폰트 설정
#   ※ 순서 주의: seaborn 스타일을 먼저 적용해야 한다.
#      sns.set_style() 이 글꼴 설정을 sans-serif 로 덮어쓰기 때문이다.
# ------------------------------------------------------------------------
sns.set_style("whitegrid")                      # ① 스타일 먼저

from matplotlib import font_manager as fm

KOREAN_FONTS = ["NanumGothic", "NanumBarunGothic", "Malgun Gothic", "AppleGothic",
                "Apple SD Gothic Neo", "Noto Sans CJK KR", "Noto Sans KR"]


def find_korean_font():
    """설치된 한글 폰트를 찾고, 없으면 Colab에서 나눔폰트를 설치한다."""
    have = {f.name for f in fm.fontManager.ttflist}
    for n in KOREAN_FONTS:
        if n in have:
            return n
    if IN_COLAB:                                # 나눔폰트 설치 후 다시 찾기
        print("한글 폰트를 설치하는 중입니다... (20초 정도 걸립니다)")
        os.system("apt-get install -y fonts-nanum > /dev/null 2>&1")
        os.system(f"{sys.executable} -m pip install -q koreanize-matplotlib")
        for path in glob.glob("/usr/share/fonts/truetype/nanum/Nanum*.ttf"):
            fm.fontManager.addfont(path)
        have = {f.name for f in fm.fontManager.ttflist}
        for n in KOREAN_FONTS:
            if n in have:
                return n
    return None

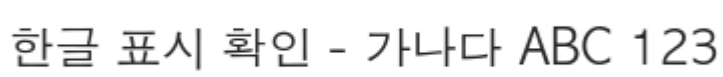

FONT = find_korean_font()                       # ② 폰트 나중에
if FONT:
    plt.rcParams["font.family"] = FONT
    plt.rcParams["font.sans-serif"] = [FONT] + list(plt.rcParams["font.sans-serif"])
plt.rcParams["axes.unicode_minus"] = False      # 음수 기호(-) 깨짐 방지
plt.rcParams["figure.figsize"] = (9, 4.5)
plt.rcParams["figure.dpi"] = 110

pd.set_option("display.max_columns", 50)
pd.set_option("display.float_format", lambda x: f"{x:,.3f}")

print("준비 완료 | pandas", pd.__version__, "| Colab:", IN_COLAB)
print("한글 폰트 :", FONT if FONT else "찾지 못함")
```

```
# ③ 자가진단 — 아래 그림에 한글이 보이면 정상입니다
plt.figure(figsize=(5, 0.9))
plt.text(0.5, 0.5, "한글 표시 확인 — 가나다 ABC 123",
         ha="center", va="center", fontsize=15)
plt.axis("off")
plt.show()
print("위 그림의 글자가 □□□ 로 보이면 [런타임 > 런타임 다시 시작] 후 이 셀을 다시 실행하세요.")
```

```
준비 완료 | pandas 2.2.3 | Colab: False
한글 폰트 : AppleGothic
```




```
위 그림의 글자가 □□□ 로 보이면 [런타임 > 런타임 다시 시작] 후 이 셀을 다시 실행하세요.
```

---

2. 데이터 가져오기

두 가지 방법이 있습니다. **① GitHub에서 바로 내려받기**(권장)가 안 되면 **② PC에서 업로드**를 쓰세요.

```
# ① GitHub에서 바로 내려받기
URL = "https://raw.githubusercontent.com/StillWork/capstone2610/main/team1_compressor.csv"
df_raw = pd.read_csv(URL)

# ② PC에서 업로드 — ①이 안 되면 위 2줄을 지우고 아래 3줄의 주석을 푸세요
# from google.colab import files
# up = files.upload()
# df_raw = pd.read_csv(list(up.keys())[0], encoding="utf-8-sig")

print("크기(행, 열):", df_raw.shape)
df_raw.head(3)
```

```
크기(행, 열): (1800, 22)
```

	record_id	timestamp	shift	compressor_id	operating_mode	load_pct	outlet_pressure_bar	air_flow_m3_
0	CP-00001	2026-07-01 00:00:00	Day	C01	LOAD	72.200	7.410	31
1	CP-00002	2026-07-01 00:00:00	Day	C02	STANDBY	0.000	6.860	2
2	CP-00003	2026-07-01 00:00:00	Day	C03	UNLOAD	24.900	7.230	11

---

3. 데이터 훑어보기

새 데이터를 받으면 **크기 → 자료형·결측 → 요약통계** 순으로 봅니다. 요약통계에서는 **불가능한 값**(음수 수량, 범위를 벗어난 값)이 있는지 확인합니다.

```
df_raw.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1800 entries, 0 to 1799
Data columns (total 22 columns):
 #   Column                     Non-Null Count  Dtype  
---  ------                     --------------  -----  
 0   record_id                  1800 non-null   object 
 1   timestamp                  1800 non-null   object 
 2   shift                      1800 non-null   object 
 3   compressor_id              1800 non-null   object 
 4   operating_mode             1800 non-null   object 
 5   load_pct                   1800 non-null   float64
 6   outlet_pressure_bar        1784 non-null   float64
 7   air_flow_m3_min            1800 non-null   float64
 8   motor_current_a            1800 non-null   float64
 9   discharge_temp_c           1785 non-null   float64
 10  oil_temp_c                 1800 non-null   float64
 11  vibration_mm_s             1779 non-null   float64
 12  dew_point_c                1800 non-null   float64
 13  filter_diff_pressure_kpa   1800 non-null   float64
 14  ambient_temp_c             1800 non-null   float64
 15  runtime_since_service_hr   1800 non-null   float64
 16  alarm_code                 1800 non-null   object 
 17  operator_check_flag        1800 non-null   object 
 18  unplanned_shutdown_flag    1800 non-null   object 
 19  downtime_min               1800 non-null   int64  
 20  maintenance_action         1800 non-null   object 
 21  next_24h_service_required  1800 non-null   object 
dtypes: float64(11), int64(1), object(10)
memory usage: 309.5+ KB
```

```
display(df_raw.describe().T.round(2))

miss = df_raw.isna().sum()
print("[결측이 있는 변수]")
print(miss[miss > 0].to_string() if (miss > 0).any() else "없음")
```

	count	mean	std	min	25%	50%	75%	max
load_pct	1,800.000	48.830	34.000	0.000	16.180	61.100	77.800	100.000
outlet_pressure_bar	1,784.000	7.130	0.190	6.390	7.010	7.130	7.260	7.720
air_flow_m3_min	1,800.000	20.550	14.420	-4.300	6.690	25.410	32.890	45.930
motor_current_a	1,800.000	34.600	11.690	11.940	23.040	38.590	44.610	56.270
discharge_temp_c	1,785.000	78.150	6.890	61.400	72.200	78.900	83.700	96.500
oil_temp_c	1,800.000	61.350	4.560	48.900	57.800	61.700	64.800	74.900
vibration_mm_s	1,779.000	1.880	0.400	0.740	1.610	1.850	2.130	3.290
dew_point_c	1,800.000	4.610	1.510	-1.300	3.600	4.600	5.600	9.300
filter_diff_pressure_kpa	1,800.000	9.300	1.560	4.800	8.220	9.220	10.280	13.800
ambient_temp_c	1,800.000	27.040	3.600	16.500	24.500	27.050	29.700	40.100
runtime_since_service_hr	1,800.000	197.420	114.390	0.000	104.950	197.450	276.080	474.700
downtime_min	1,800.000	1.130	10.640	0.000	0.000	0.000	0.000	136.000

```
[결측이 있는 변수]
outlet_pressure_bar    16
discharge_temp_c       15
vibration_mm_s         21
```

---

4. 데이터 정제

항목	처리
공기유량 음수 (영점오차)	**0으로 보정**
토출압력·토출온도·진동 결측	**운전모드별 중앙값**으로 채움 (모드마다 정상범위가 다르므로)

```
df = df_raw.copy()

# ① 공기유량 음수 → 0 (물리적으로 불가능한 값, 영점오차로 판단)
print("공기유량 음수:", int((df["air_flow_m3_min"] < 0).sum()), "건 → 0으로 보정")
df.loc[df["air_flow_m3_min"] < 0, "air_flow_m3_min"] = 0

# ② 센서 결측 → 같은 운전모드의 중앙값으로 채움
for c in ["outlet_pressure_bar", "discharge_temp_c", "vibration_mm_s"]:
    n = int(df[c].isna().sum())
    df[c] = df[c].fillna(df.groupby("operating_mode")[c].transform("median"))
    print(f"{c} 결측 {n}건 → 운전모드별 중앙값으로 채움")

df["timestamp"] = pd.to_datetime(df["timestamp"])
print("\n남은 결측:", int(df.isna().sum().sum()), "건")
```

```
공기유량 음수: 174 건 → 0으로 보정
outlet_pressure_bar 결측 16건 → 운전모드별 중앙값으로 채움
discharge_temp_c 결측 15건 → 운전모드별 중앙값으로 채움
vibration_mm_s 결측 21건 → 운전모드별 중앙값으로 채움

남은 결측: 0 건
```

---

5. 목표변수 — 찾아내고 싶은 사건은 얼마나 드문가

목표변수는 `next_24h_service_required` 입니다. 정비필요(Y) 을 **1**, 나머지를 **0** 으로 둡니다.

양성이 드물면 **"전부 음성"이라고만 답해도 정확도가 매우 높습니다.** 그래서 이 데이터에서는 **정확도를 믿으면 안 됩니다.**

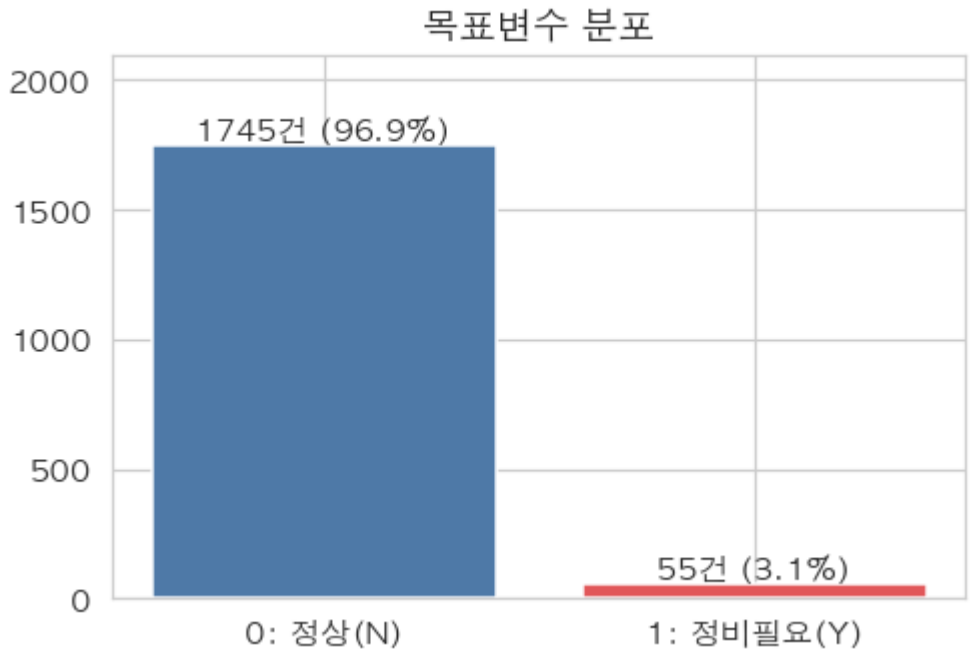
```python
y = (df['next_24h_service_required'] == 'Y').astype(int)          # 1 = 정비필요(Y)
df["target"] = y

print(df["next_24h_service_required"].value_counts().to_string())
print(f"\n양성(1) 비율            : {y.mean()*100:.2f}%")
print(f"'전부 음성' 모델의 정확도 : {(1-y.mean())*100:.2f}%  ← 이 숫자를 넘어야 의미가 있다")

cnt = y.value_counts().sort_index()
plt.figure(figsize=(5, 3.2))
plt.bar(["0: 정상(N)", "1: 정비필요(Y)"], cnt.values, color=["#4e79a7", "#e15759"])
for i, v in enumerate(cnt.values):
    plt.text(i, v, f"{v}건 ({v/len(y)*100:.1f}%)", ha="center", va="bottom")
plt.ylim(0, cnt.values.max() * 1.2)
plt.title("목표변수 분포")
plt.show()
```

```
next_24h_service_required
N    1745
Y      55

양성(1) 비율            : 3.06%
'전부 음성' 모델의 정확도 : 96.94%  ← 이 숫자를 넘어야 의미가 있다
```



---

6. 탐색 ① 그룹별 양성률 — 어느 조건에서 문제가 많은가

반드시 표본 수를 함께 보세요. "10건 중 2건(20%)"과 "400건 중 40건(10%)"은 비율만 보면 앞이 2배 높지만, 앞은 1건만 달라져도 크게 흔들립니다.

```python
def group_rate(col):
    g = df.groupby(col)["target"]
    t = pd.DataFrame({"전체건수": g.size(), "양성건수": g.sum().astype(int)})
    t["양성률(%)"] = (t["양성건수"] / t["전체건수"] * 100).round(2)
    return t


GROUPS = ["compressor_id", "operating_mode", "shift", "alarm_code"]
for c in GROUPS:
    print(f"===== {c} =====")
    display(group_rate(c))
```

```
===== compressor_id =====
```

compressor_id	전체건수	양성건수	양성률(%)
C01	450	14	3.110
C02	450	13	2.890
C03	450	17	3.780
C04	450	11	2.440

```
===== operating_mode =====
```

operating_mode	전체건수	양성건수	양성률(%)
LOAD	1047	36	3.440
STANDBY	368	12	3.260
UNLOAD	385	7	1.820

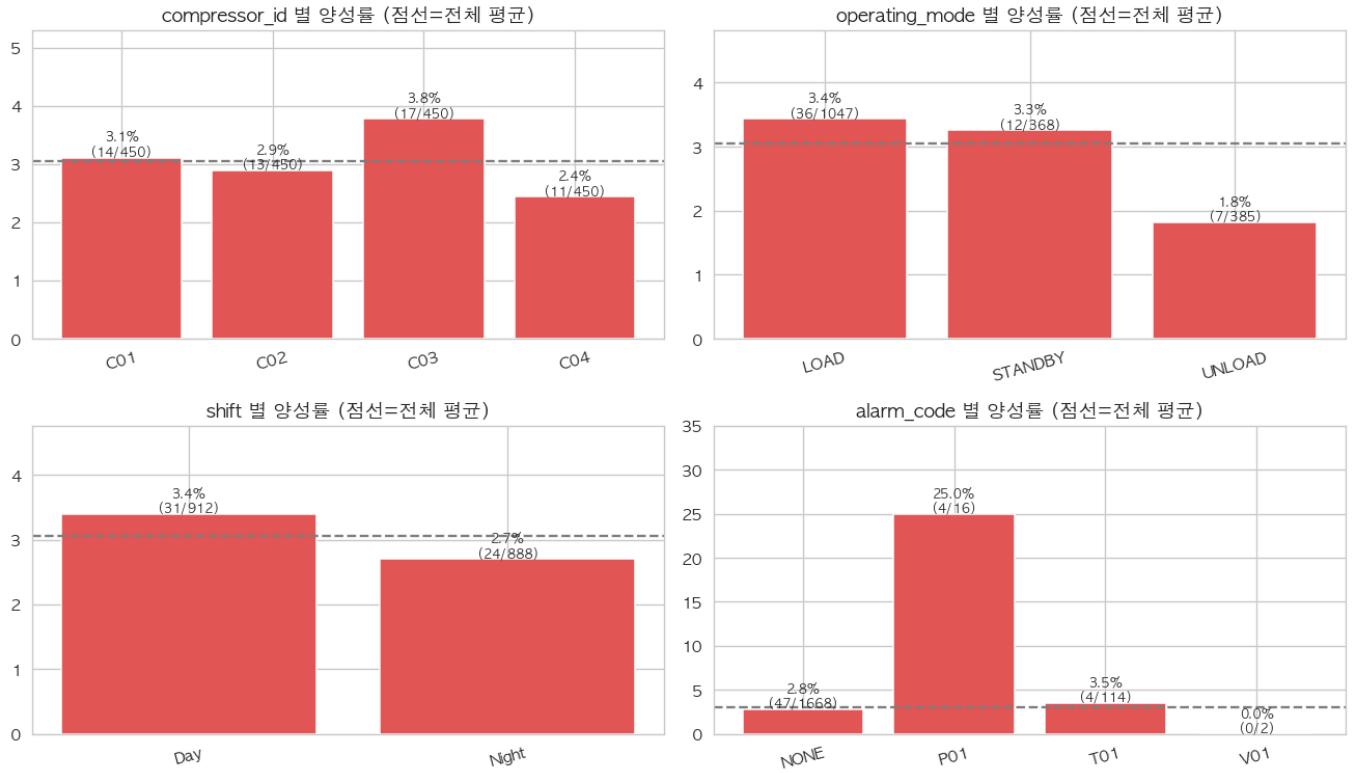
```
===== shift =====
```

shift	전체건수	양성건수	양성률(%)
Day	912	31	3.400
Night	888	24	2.700

```
===== alarm_code =====
```

alarm_code	전체건수	양성건수	양성률(%)
NONE	1668	47	2.820
P01	16	4	25.000
T01	114	4	3.510
V01	2	0	0.000

```
overall = df["target"].mean() * 100
fig, axes = plt.subplots(2, 2, figsize=(12, 7))
for ax, c in zip(axes.ravel(), GROUPS):
    t = group_rate(c)
    ax.bar(t.index.astype(str), t["양성률(%)"], color="#e15759")
    ax.axhline(overall, color="gray", ls="--")
    for i, (n, m, r) in enumerate(zip(t["전체건수"], t["양성건수"], t["양성률(%)"])):
        ax.text(i, r, f"{r:.1f}%\n({m}/{n})", ha="center", va="bottom", fontsize=8)
    ax.set_title(f"{c} 별 양성률 (점선=전체 평균)")
    ax.set_ylim(0, max(t["양성률(%)"].max() * 1.4, 1))
    ax.tick_params(axis="x", rotation=15)
plt.tight_layout()
plt.show()
```



---

7. 탐색 ② 교차표 — 두 조건을 겹쳐서 보기

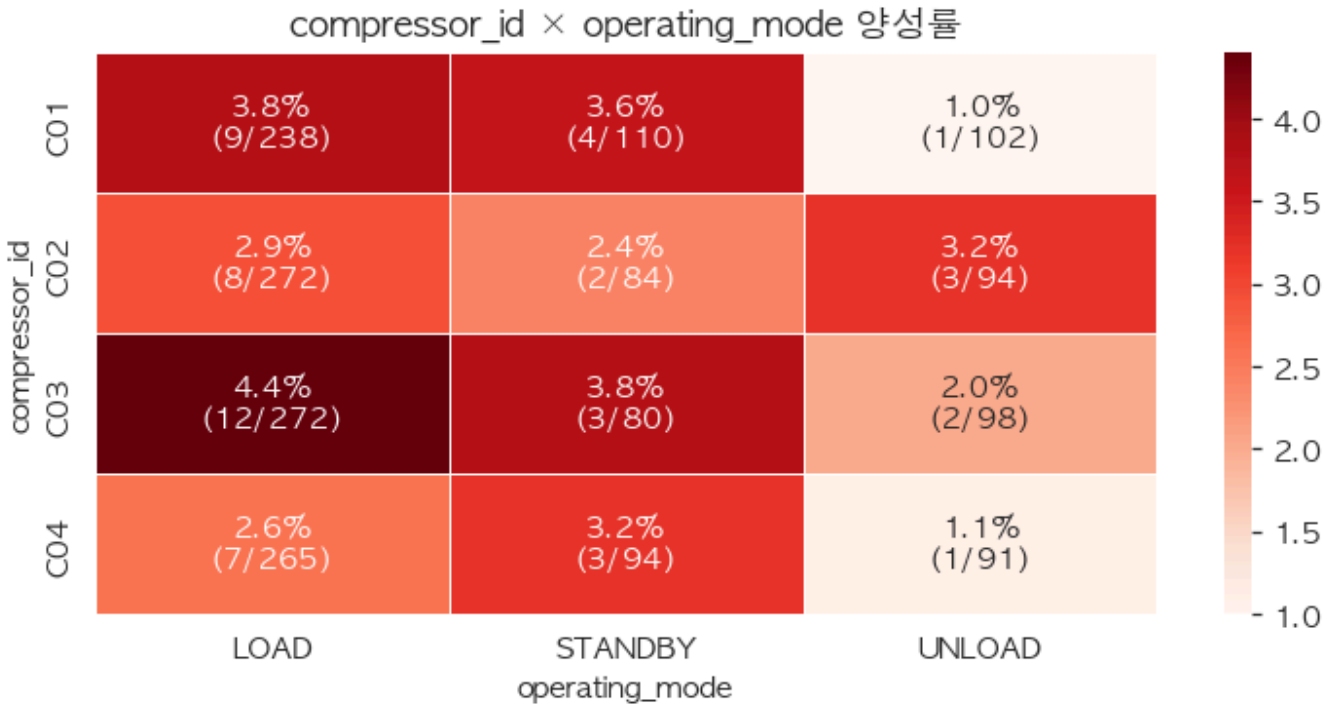
그룹을 하나씩 볼 때 보이는 차이가, 사실은 **다른 변수 때문**일 수 있습니다(**교란**). `compressor_id` 와 `operating_mode` 를 겹쳐서 확인합니다.

```python
GX, GY = "compressor_id", "operating_mode"

cnt = pd.crosstab(df[GX], df[GY])
pos = pd.crosstab(df[GX], df[GY], values=df["target"], aggfunc="sum").fillna(0)
rate = (pos / cnt * 100).round(1)

plt.figure(figsize=(8, 3.4))
annot = rate.astype(str) + "%\n(" + pos.astype(int).astype(str) + "/" + cnt.astype(str) + ")"
sns.heatmap(rate, annot=annot, fmt="", cmap="Reds", linewidths=0.5)
plt.title(f"{GX} × {GY} 양성률")
plt.show()

print("칸의 건수가 적으면(10건 미만) 그 칸의 비율은 참고만 한다.")
```



칸의 건수가 적으면(10건 미만) 그 칸의 비율은 참고만 한다.

---

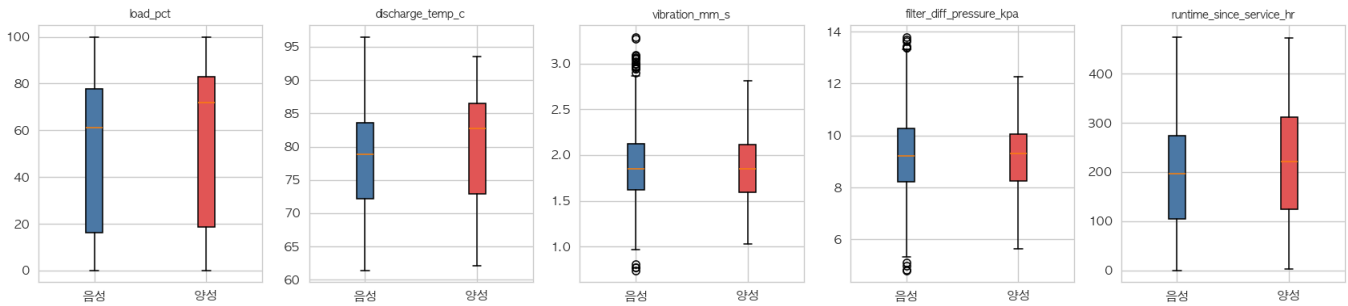
8. 탐색 ③ 수치형 변수 비교 — 양성/음성은 무엇이 다른가

상자가 많이 겹치면 **그 변수 하나로는 구분할 수 없다**는 뜻입니다. 제조 데이터는 대부분 이렇고, 그래서 **여러 변수를 조합하는 머신러닝**이 필요합니다.

```
NUM_VIZ = [c for c in [
    "load_pct", "discharge_temp_c", "vibration_mm_s", "filter_diff_pressure_kpa",
    "runtime_since_service_hr"
] if c in df.columns]

fig, axes = plt.subplots(1, len(NUM_VIZ), figsize=(3.1 * len(NUM_VIZ), 3.6))
for ax, c in zip(np.atleast_1d(axes).ravel(), NUM_VIZ):
    data = [df.loc[df["target"] == 0, c].dropna(), df.loc[df["target"] == 1, c].dropna()]
    bp = ax.boxplot(data, patch_artist=True)
    for patch, color in zip(bp["boxes"], ["#4e79a7", "#e15759"]):
        patch.set_facecolor(color)
    ax.set_xticks([1, 2])
    ax.set_xticklabels(["음성", "양성"])
    ax.set_title(c, fontsize=9)
plt.tight_layout()
plt.show()

cmp = pd.DataFrame({"음성 평균": df[df["target"] == 0][NUM_VIZ].mean(),
                    "양성 평균": df[df["target"] == 1][NUM_VIZ].mean()})
cmp["차이"] = cmp["양성 평균"] - cmp["음성 평균"]
display(cmp.round(3))
```



	음성 평균	양성 평균	차이
load_pct	48.638	55.024	6.386
discharge_temp_c	78.101	80.055	1.953
vibration_mm_s	1.878	1.882	0.005
filter_diff_pressure_kpa	9.294	9.333	0.038
runtime_since_service_hr	196.517	226.156	29.639

---

9. 탐색 ④ 산점도와 상관관계

산점도는 점 하나가 기록 하나입니다. 색으로 양성/음성을 구분하면 **문제가 어느 영역에 모여 있는지** 눈으로 확인할 수 있습니다.

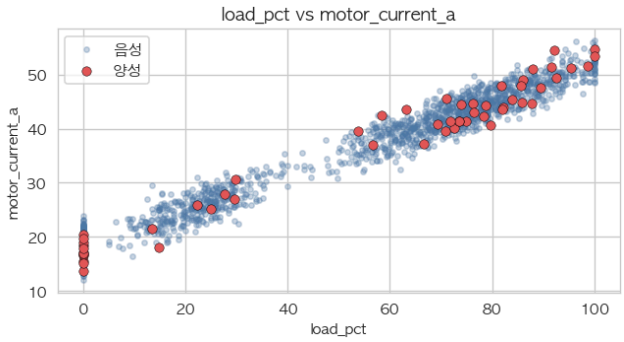
> 부하율이 올라가면 모터전류도 같이 올라가는지 - 센서 정합성 점검

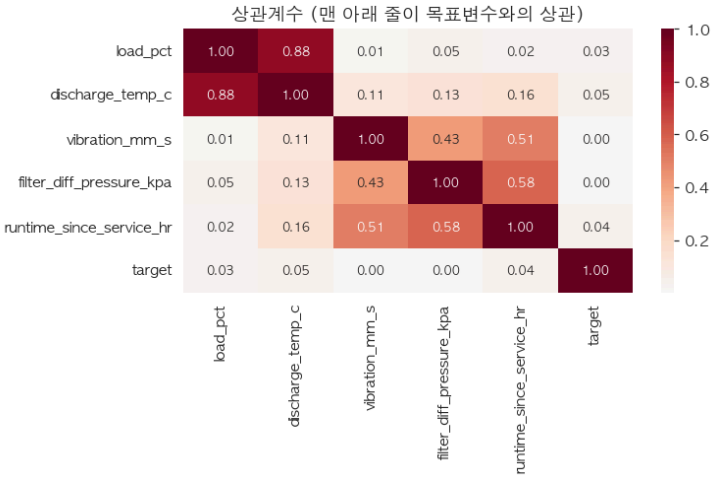
상관계수는 **직선 관계의 강도**(-1~+1)입니다. 세 가지를 기억하세요: 곡선 관계는 못 잡고, **상관은 인과가 아니며**, 이상치에 민감합니다.

```
X1, Y1 = "load_pct", "motor_current_a"

fig, axes = plt.subplots(1, 2, figsize=(13, 4.6))
neg, pos_ = df[df["target"] == 0], df[df["target"] == 1]
axes[0].scatter(neg[X1], neg[Y1], s=12, alpha=0.3, c="#4e79a7", label="음성")
axes[0].scatter(pos_[X1], pos_[Y1], s=38, c="#e15759", edgecolors="black",
                linewidths=0.3, label="양성")
axes[0].set_xlabel(X1)
axes[0].set_ylabel(Y1)
axes[0].set_title(f"{X1} vs {Y1}")
axes[0].legend()

corr = df[NUM_VIZ + ["target"]].corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", center=0, ax=axes[1],
            annot_kws={"size": 8})
axes[1].set_title("상관계수 (맨 아래 줄이 목표변수와의 상관)")
plt.tight_layout()
plt.show()
```





---

10. 입력변수 고르기 — 데이터 누출 막기

판단 시점(교대 종료 직전 (지금까지의 센서값은 알고, 앞으로 24시간에 일어날 일은 모르는 시점))에 **이미 알 수 있는 변수만** 사용합니다.

쓸 수 없는 변수 (결과가 나온 뒤에 생기는 값)

변수	이유
`unplanned_shutdown_flag`	비계획정지 여부. 사고가 난 뒤에야 확정되는 결과다
`downtime_min`	정지시간. 정지가 발생한 뒤 집계된다
`maintenance_action`	정비조치 내용. 정비를 했다는 기록이므로 정답과 거의 같다
`record_id`	단순 식별번호. 의미 있는 정보가 없다

> 성능이 비현실적으로 높으면(정확도 99%, AUC 0.99) **먼저 누출을 의심하세요.** 실무에서 가장 흔한 실패 원인입니다.

```python
CAT = [c for c in [
    "shift", "compressor_id", "operating_mode", "alarm_code", "operator_check_flag"
] if c in df.columns]      # 범주형 입력
NUM = [c for c in [
    "load_pct", "outlet_pressure_bar", "air_flow_m3_min", "motor_current_a",
    "discharge_temp_c", "oil_temp_c", "vibration_mm_s", "dew_point_c",
    "filter_diff_pressure_kpa", "ambient_temp_c", "runtime_since_service_hr"
] if c in df.columns]      # 수치형 입력
LEAK = ["unplanned_shutdown_flag", "downtime_min", "maintenance_action", "record_id"]

print(f"입력 변수: 범주형 {len(CAT)}개 + 수치형 {len(NUM)}개 = {len(CAT)+len(NUM)}개")
print("범주형:", CAT)
print("수치형:", NUM)
```

```
입력 변수: 범주형 5개 + 수치형 11개 = 16개
범주형: ['shift', 'compressor_id', 'operating_mode', 'alarm_code', 'operator_check_flag']
수치형: ['load_pct', 'outlet_pressure_bar', 'air_flow_m3_min', 'motor_current_a', 'discharge_tem
p_c', 'oil_temp_c', 'vibration_mm_s', 'dew_point_c', 'filter_diff_pressure_kpa', 'ambient_temp
_c', 'runtime_since_service_hr']
```

---

11. 전처리 — 원핫인코딩 · 분리 · 스케일링

처리	이유
원핫인코딩	모델은 문자를 모른다. 범주마다 0/1 열을 만든다
학습/평가 분리	학습에 쓴 데이터로 평가하면 항상 성적이 좋다(시험문제를 미리 본 셈)
스케일링	변수마다 단위·범위가 달라서 큰 숫자 변수가 과대평가된다

순서가 중요합니다. 스케일러는 **학습 데이터에서만** 기준(평균·표준편차)을 배우고, 평가 데이터에는 그 기준을 **적용만** 해야 합니다. 전체 데이터로 먼저 스케일링하면 평가 데이터의 정보가 학습에 새어 들어갑니다(**데이터 누출**).

```python
# ① 원핫인코딩 + 입력행렬 만들기
X = pd.concat([df[NUM], pd.get_dummies(df[CAT])], axis=1)
print("원핫인코딩 후 열 개수:", X.shape[1])

# ② 학습 75% / 평가 25% 로 분리 (stratify=y 로 양성 비율을 양쪽에 똑같이 맞춘다)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

# ③ 스케일링 – 반드시 학습 데이터에서만 fit !
sc = StandardScaler()
X_train = X_train.copy()
X_test = X_test.copy()
X_train[NUM] = sc.fit_transform(X_train[NUM])    # 학습: 기준을 배우고 변환
X_test[NUM] = sc.transform(X_test[NUM])          # 평가: 배운 기준으로 변환만

print(f"학습 {len(X_train)}건 (양성 {int(y_train.sum())}) / "
      f"평가 {len(X_test)}건 (양성 {int(y_test.sum())})")
X_train.head(3)
```

```
원핫인코딩 후 열 개수: 26
학습 1350건 (양성 41) / 평가 450건 (양성 14)
```

	load_pct	outlet_pressure_bar	air_flow_m3_min	motor_current_a	discharge_temp_c	oil_temp_c	vibr
319	-1.014	0.205	-1.094	-0.842	-0.836	-2.043	
1389	1.340	1.616	1.324	1.330	1.492	1.180	
1476	0.945	-0.012	1.062	0.952	0.692	1.070	

---

12. 모델 3종 학습과 성능평가

모델	방식	특징
로지스틱 회귀	변수에 가중치를 곱해 더한 뒤 확률로 변환	해석이 쉽다
의사결정나무	스무고개처럼 질문을 이어가며 나눔	그림으로 볼 수 있다
랜덤포레스트	나무 300그루의 다수결	성능이 가장 좋은 편

`class_weight="balanced"` **를 반드시 넣으세요.** 양성이 드물면 모델은 "전부 음성"이라고 답하는 것이 손해가 적다고 판단합니다. 이 옵션은 **드문 쪽을 틀렸을 때 벌점을 크게** 주어 그것을 막습니다.

혼동행렬과 지표

	예측: 음성	예측: 양성
실제 음성	TN	**FP (헛경보)**
실제 양성	**FN (놓침)**	TP

- **Recall** = TP/(TP+FN) : 실제 양성 중 몇 %를 잡았나 → **놓치면 안 될 때 중요**
- **Precision** = TP/(TP+FP) : 경보 중 몇 %가 진짜였나 → **헛경보 비용이 클 때 중요**
- **ROC-AUC** : 임계값과 무관하게 "양성에 더 높은 점수를 줄 확률". 0.5=무작위

```python
models = {
    "로지스틱회귀": LogisticRegression(max_iter=2000, class_weight="balanced"),
    "의사결정나무": DecisionTreeClassifier(max_depth=4, min_samples_leaf=20,
                                          class_weight="balanced", random_state=42),
    "랜덤포레스트": RandomForestClassifier(n_estimators=300, max_depth=8,
                                          min_samples_leaf=5, class_weight="balanced",
                                          random_state=42),
}

rows = []
for name, m in models.items():
    m.fit(X_train, y_train)
    proba = m.predict_proba(X_test)[:, 1]
    pred = m.predict(X_test)
    tn, fp, fn, tp = confusion_matrix(y_test, pred, labels=[0, 1]).ravel()
    rows.append({"모델": name, "TP": tp, "FN": fn, "FP": fp, "TN": tn,
                 "Recall": recall_score(y_test, pred, zero_division=0),
                 "Precision": precision_score(y_test, pred, zero_division=0),
                 "ROC_AUC": roc_auc_score(y_test, proba)})
    print(f"===== {name} =====")
    print(classification_report(y_test, pred, zero_division=0))

result = pd.DataFrame(rows).set_index("모델")
display(result.round(3))

base = 1 - y_test.mean()
print(f"참고: '전부 음성' 모델의 정확도 = {base*100:.1f}% (이 숫자는 Recall 0 이다)")
```

```
===== 로지스틱회귀 =====
              precision    recall  f1-score   support

           0       0.98      0.77      0.86       436
           1       0.07      0.57      0.13        14

    accuracy                           0.76       450
   macro avg       0.53      0.67      0.50       450
weighted avg       0.95      0.76      0.84       450

===== 의사결정나무 =====
              precision    recall  f1-score   support

           0       0.98      0.82      0.90       436
           1       0.09      0.57      0.16        14

    accuracy                           0.82       450
   macro avg       0.54      0.70      0.53       450
weighted avg       0.96      0.82      0.87       450

===== 랜덤포레스트 =====
              precision    recall  f1-score   support

           0       0.98      0.99      0.99       436
           1       0.55      0.43      0.48        14

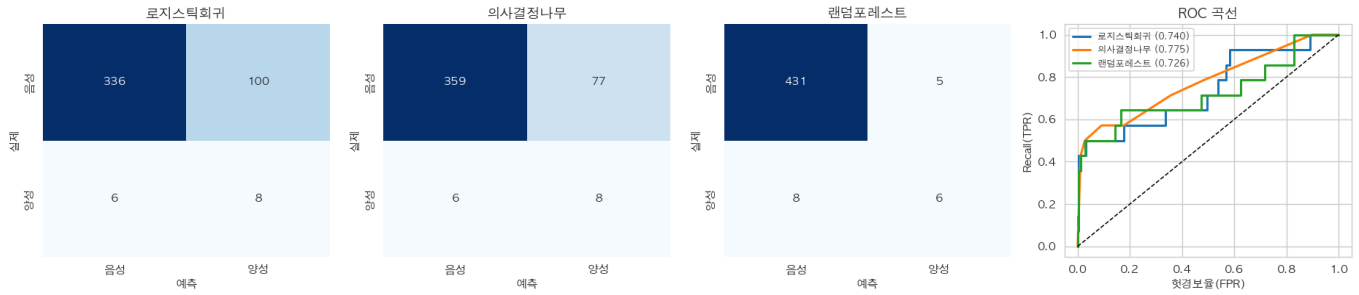
    accuracy                           0.97       450
   macro avg       0.76      0.71      0.73       450
weighted avg       0.97      0.97      0.97       450
```

모델	TP	FN	FP	TN	Recall	Precision	ROC_AUC
로지스틱회귀	8	6	100	336	0.571	0.074	0.740
의사결정나무	8	6	77	359	0.571	0.094	0.775
랜덤포레스트	6	8	5	431	0.429	0.545	0.726

참고: '전부 음성' 모델의 정확도 = 96.9% (이 숫자는 Recall 0 이다)

```
# 혼동행렬 + ROC 곡선
fig, axes = plt.subplots(1, 4, figsize=(17, 3.8))
for ax, (name, m) in zip(axes[:3], models.items()):
    cm = confusion_matrix(y_test, m.predict(X_test), labels=[0, 1])
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, ax=ax,
                xticklabels=["음성", "양성"], yticklabels=["음성", "양성"])
    ax.set_title(name)
    ax.set_xlabel("예측")
    ax.set_ylabel("실제")

for name, m in models.items():
    proba = m.predict_proba(X_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, proba)
    axes[3].plot(fpr, tpr, lw=2, label=f"{name} ({roc_auc_score(y_test, proba):.3f})")
axes[3].plot([0, 1], [0, 1], "k--", lw=1)
axes[3].set_xlabel("헛경보율(FPR)")
axes[3].set_ylabel("Recall(TPR)")
axes[3].set_title("ROC 곡선")
axes[3].legend(fontsize=8)
plt.tight_layout()
plt.show()
```



---

13. 임계값 조정 — 모델을 바꾸지 않고 운영점 바꾸기

모델은 0/1이 아니라 **확률**을 출력합니다. `predict()` 는 0.5를 기준으로 자르는데, 이 기준은 **바꿀 수 있습니다.**

- 임계값을 **낮추면**: 경보가 늘고 → Recall↑, Precision↓
- 임계값을 **높이면**: 경보가 줄고 → Recall↓, Precision↑

현장 제약을 먼저 정하고 임계값을 고르는 것이 올바른 순서입니다.

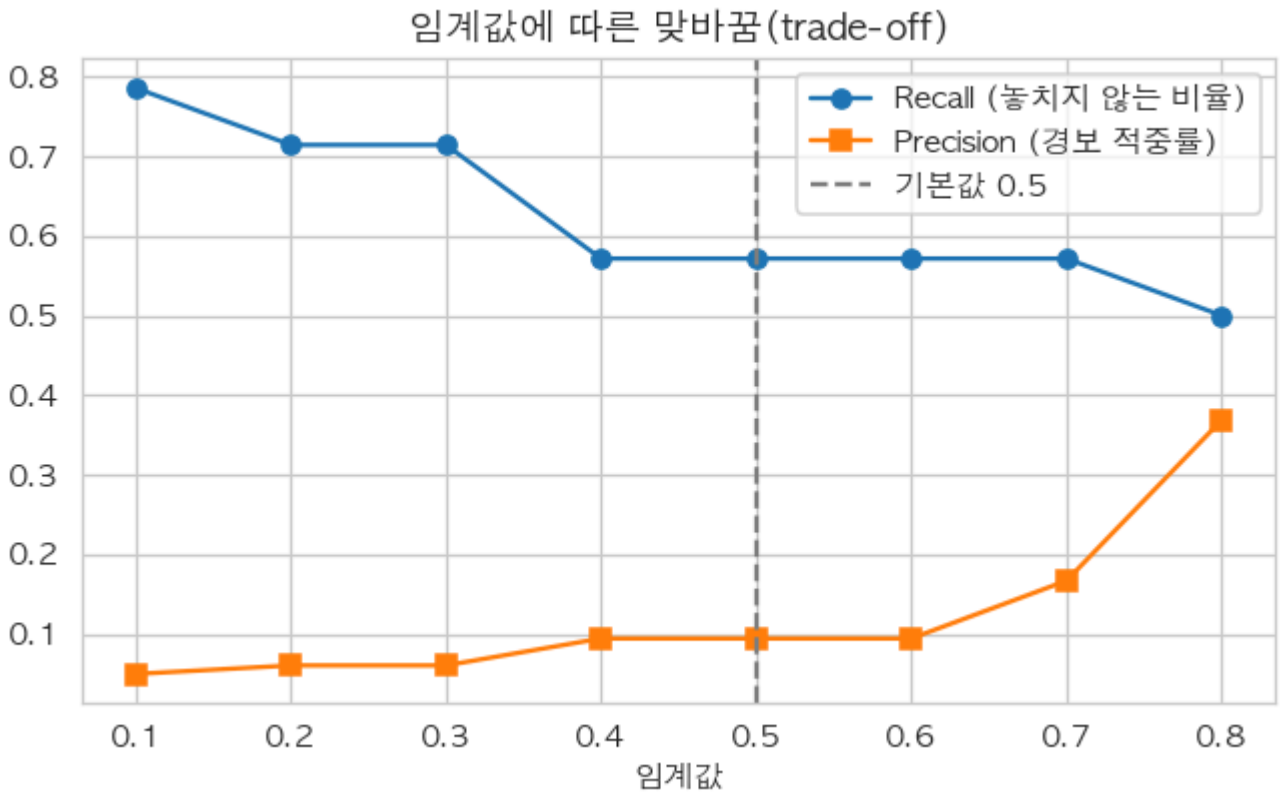
```python
best_name = result["ROC_AUC"].idxmax()          # 임계값과 무관한 지표로 기준 모델 선택
best = models[best_name]
proba = best.predict_proba(X_test)[:, 1]
print("기준 모델:", best_name)

rows = []
for th in [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]:
    pred = (proba >= th).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, pred, labels=[0, 1]).ravel()
    rows.append({"임계값": th, "경보건수": int(pred.sum()), "TP(잡음)": tp,
                 "FN(놓침)": fn, "FP(헛경보)": fp,
                 "Recall": recall_score(y_test, pred, zero_division=0),
                 "Precision": precision_score(y_test, pred, zero_division=0)})
th_tbl = pd.DataFrame(rows)
display(th_tbl.round(3))

plt.figure(figsize=(7, 3.8))
plt.plot(th_tbl["임계값"], th_tbl["Recall"], "o-", label="Recall (놓치지 않는 비율)")
plt.plot(th_tbl["임계값"], th_tbl["Precision"], "s-", label="Precision (경보 적중률)")
plt.axvline(0.5, color="gray", ls="--", label="기본값 0.5")
plt.xlabel("임계값")
plt.legend()
plt.title("임계값에 따른 맞바꿈(trade-off)")
plt.show()
```

기준 모델: 의사결정나무

	임계값	경보건수	TP(잡음)	FN(놓침)	FP(헛경보)	Recall	Precision
0	0.100	222	11	3	211	0.786	0.050
1	0.200	166	10	4	156	0.714	0.060
2	0.300	166	10	4	156	0.714	0.060
3	0.400	85	8	6	77	0.571	0.094
4	0.500	85	8	6	77	0.571	0.094
5	0.600	85	8	6	77	0.571	0.094
6	0.700	48	8	6	40	0.571	0.167
7	0.800	19	7	7	12	0.500	0.368



---

14. 특성중요도 — 모델은 무엇을 보고 판단했나

랜덤포레스트는 "이 변수로 나눴을 때 양성/음성이 얼마나 깔끔하게 갈리는가"를 기준으로 분기합니다. 그 기여도를 모두 합한 것이 **특성중요도**입니다(합 = 1).

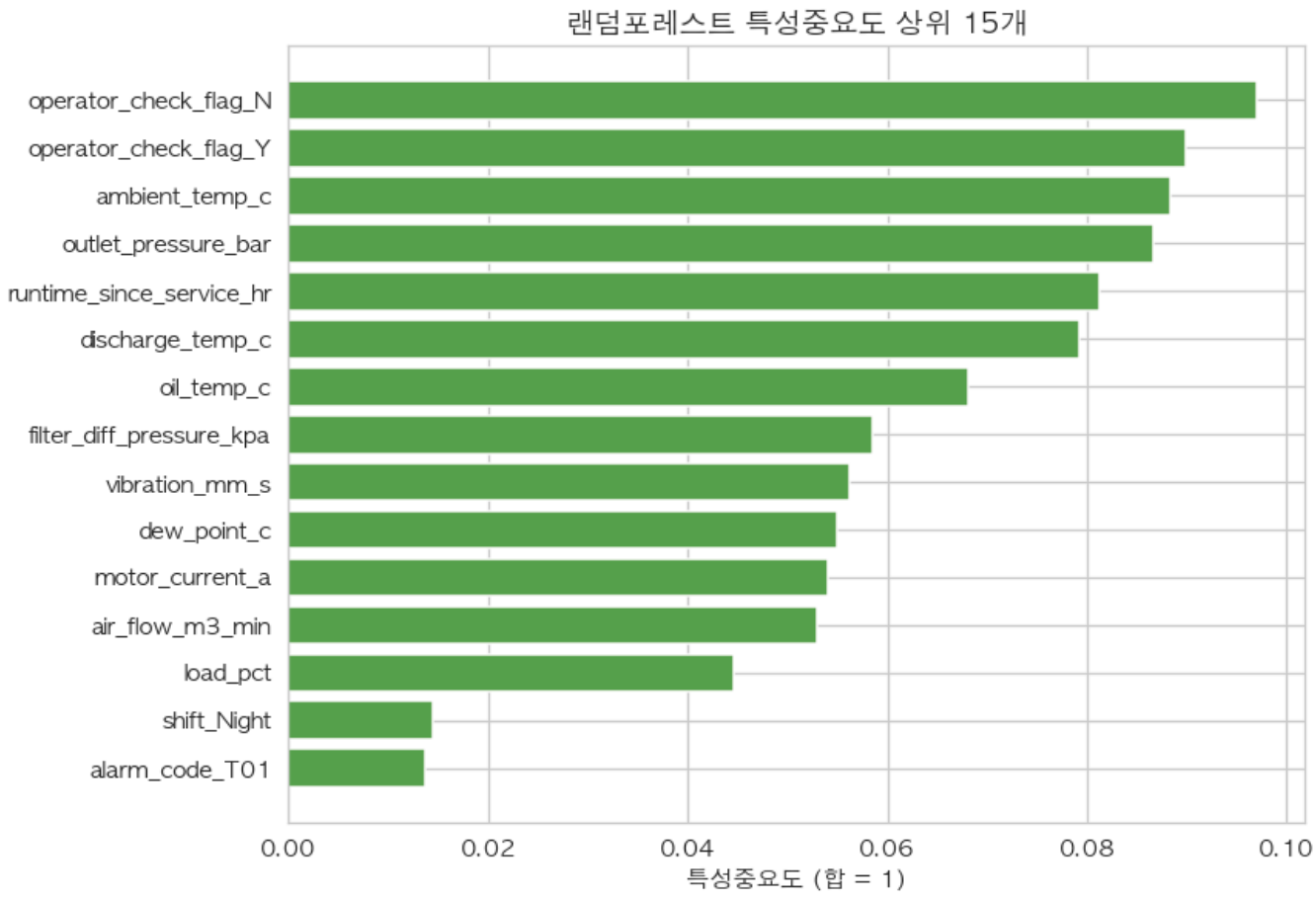
> **주의**: 특성중요도는 **"모델이 많이 참고한 변수"** 이지 **"현실의 원인"** 이 아닙니다. 인과를 주장하려면 현장에서 조건을 바꿔 보는 확인이 필요합니다.

```
rf = models["랜덤포레스트"]
imp = pd.DataFrame({"변수": X_train.columns,
                    "중요도": rf.feature_importances_}).sort_values("중요도", ascending=False)
display(imp.head(10).round(4))

top = imp.head(15)[::-1]
plt.figure(figsize=(8, 5.5))
plt.barh(top["변수"], top["중요도"], color="#59a14f")
plt.xlabel("특성중요도 (합 = 1)")
plt.title("랜덤포레스트 특성중요도 상위 15개")
plt.tight_layout()
plt.show()

print("상위 5개 변수가 전체 중요도의", f"{imp.head(5)['중요도'].sum()*100:.0f}% 를 차지합니다.")
```

	변수	중요도
24	operator_check_flag_N	0.097
25	operator_check_flag_Y	0.090
9	ambient_temp_c	0.088
1	outlet_pressure_bar	0.087
10	runtime_since_service_hr	0.081
4	discharge_temp_c	0.079
5	oil_temp_c	0.068
8	filter_diff_pressure_kpa	0.059
6	vibration_mm_s	0.056
7	dew_point_c	0.055




```
상위 5개 변수가 전체 중요도의 44% 를 차지합니다.
```

15. 정리

이 데이터에서 확인한 것

- 정비필요(Y)는 1,800건 중 55건(3.06%)뿐이다. **정확도 대신 Recall·Precision·F1**을 본다.
- 압축기별 Y 건수는 C03이 17건으로 가장 많고 C04가 11건으로 가장 적다. 다만 차이가 크지 않으므로 "C03이 불량 설비"라고 단정하기 전에 **운전조건(부하·운전모드 구성)**을 봐야 한다.
- Y 그룹은 N 그룹보다 평균 부하율·모터전류·토출온도·정비 후 누적운전시간이 다소 높다. 그러나 **단일 변수로는 겹치는 구간이 넓어** 깔끔하게 나뉘지 않는다.
- 알람코드는 대부분 NONE(1,668건)이고, 알람이 울린 구간에도 정비필요가 아닌 경우가 많다. **알람만으로는 예지보전이 안 된다**는 근거다.
- 결측은 토출압력 16건, 토출온도 15건, 진동 21건이다. 무조건 삭제하기 전에 **어떤 압축기·모드·시간대에 몰려 있는지** 확인해야 한다.
- `timestamp` + `compressor_id` 조합에 중복(72건)이 있다. 시계열 분석 전에 반드시 확인한다.

결론과 다음 단계

- 정비필요가 3%뿐이라 **정확도는 의미가 없다.** Recall(놓치지 않는 비율)과 "하루에 몇 대를 점검할 수 있는가"를 함께 본다.
- 운전모드(LOAD/UNLOAD/STANDBY)를 빼놓고 센서값을 비교하면 틀린 결론이 나온다.
- 다음 단계: 설비별 **시간에 따른 변화량**(온도 상승 추세, 진동 증가 추세)을 변수로 만들면 성능이 올라갈 가능성이 크다. 지금은 한 시점의 값만 쓰고 있다.

공통으로 기억할 5가지

#	원칙
1	**정확도만 보지 않는다** — 양성이 드물면 아무것도 안 해도 95%가 넘는다
2	**놓침(FN)과 헛경보(FP)의 비용은 다르다** — 어느 쪽이 비싼지는 현장이 정한다
3	**성능이 너무 좋으면 누출을 의심한다** — 결과 변수를 넣지 않았는지 확인
4	**임계값은 운영 정책이다** — 모델을 바꾸지 않고도 경보 수를 조절할 수 있다
5	**상관은 인과가 아니다** — 특성중요도는 원인이 아니라 단서다

연습문제 / 생각해 볼 질문

아래 문항은 정답을 맞히는 시험이 아니라, **발표자료에 쓸 문장을 만드는 연습**입니다. 코드를 조금씩 바꿔 보고, 결과를 한 문장으로 정리해 보세요.

1. LOAD 구간만 골라(`df[df['operating_mode']=='LOAD']`) 압축기별 토출온도 박스플롯을 다시 그려 보세요. 전체 데이터로 그린 것과 결론이 달라집니까?
2. `air_flow_m3_min` 이 음수인 행을 모두 출력하고, 운전모드 분포를 확인하세요. 이 값을 0으로 바꿀까요, 결측으로 둘까요? 선택과 근거를 적으세요.
3. `split_mode` 를 'random' 과 'time' 으로 바꿔 실행하고, 두 경우의 테스트 성능 차이를 적으세요. 어느 쪽이 현장 상황에 더 가깝습니까?
4. `alarm_code` 와 `operator_check_flag` 를 특성에서 제외하고 다시 학습해 성능 변화를 확인하세요. 알람 없이도 예측이 되나요?